第1周-习题课1

2018年5月 星期二 4:06

一、OJ使用和习题课

&习题索引：

1.栈；（栈+循秩访问）

2.队列；（队列+循秩访问）

3.二叉树。（二叉排序树、前序遍历、后序遍历）

@输入输出外挂总结（233）

来自 <https://blog.csdn.net/f_zyj/article/details/51473493>

1.习题课安排

周三、周五（徐良钦），周四（王聿(yù)中）。【讲课风格可能不太一样】

可在直播间下的讨论区提问。附加题不计算分数。无ddl限制。

补交——>得分降低（85%）。

当天讲解题目提交时间在讲习题课之前。（之后将尽量早地上传）

&&第0周栈排序用到了**C++11**新特性，今后将避免。

&&代码UTF-8编码，可能出现(中文)乱码。需要设置IDE编码格式，网上搜索解决即可。

2.OJ介绍和提交示范

可参考之前发的OJ使用教程。

- 打开对应课程，找到相应习题
- 点击Judge，选择文件并提交

! **【特别提示】：不需要Honor Code，不需要Report，也不需要mark！！！**

支持C、C++、Py2、Py3、Java。除了C/C++，其他语言不统计内存。

- （提交Py3务必：将“#!/usr/bin/env python3”加在代码第一行，否则出现错误）
- （提交Java务必：主类名必须是Main）

&&不提倡具体实现的竞争，无意义，提交正确即可。

&&Python、Java运行较慢，无须太在意。

3.学员提交情况

通过人数：100-92-62。

4.其它

讨论区提问要求

- **邮箱**（昵称/名字），**代码**（可选，贴图[需要缩进]）。

提供代码

- 不关心输入输出格式，专注算法本身
- 获得一些解题提示（当然可以选择不用）

如何设计测试样例？

一个大概的程序化思路：

1. 确保题目没看错
2. 确保数据范围没看错
3. 确保自己的算法是正确的
4. 至少样例过了
5. [可以跳过]自己生成“特殊数据”，比如边界数据（小边界和大边界）
6. [可以跳过]寻求题解、帮助

为什么不直接介绍最优解法

- 研究低效、原始的解法，从而开发更高效的算法，验证新解法的正确性

不同实现方式

- 呈现更优美的实现方式

内置数据类型对应表：

C	C++	Java	Python
int long long	int long long	int/Integer long/Long	int long
double char *	double string	double String	double str
...? QAQ	vector set	List HashMap	list()/[] set()/{}
_(x3」∠)_	map	HashMap	dict()

二、习题讲解

1.栈

描述

实现一个栈，完成以下功能：

1. 入栈
2. 出栈
3. 询问栈中位置Y是谁

一开始栈为空。栈中的位置从1开始（即栈底位置为1）。

输入

第一行一个整数n，表示操作个数。

接下来n行，每行第一个数字表示操作（见描述）：

- 若为数字1，则接下来有一串字符串X，表示将X压入栈中。
- 若为数字2，表示弹出栈顶（保证栈非空），并输出出栈的这个人。
- 若为数字3，则接下来有一个整数Y，表示询问栈中位置Y是谁（保证位置Y合法），并输出名字。

输出

将所有操作2和操作3输出，一行一个。

输入样例

```
11
1 a
1 b
1 c
3 1
3 2
3 3
2
1 d
3 1
3 2
3 3
```

输出样例

```
a
b
c
c
a
b
d
```

限制

对于30%的数据，1 ≤ n ≤ 2000；

对于另30%的数据，没有操作3；

对于100%的数据，**1 ≤ n ≤ 100000**。

数据中出现的字符串只包含26个小写字母（无空格等分隔符），且长度不超过15。

字符串有可能重复。正如现实中可能有重名一样。

时间：2 sec

空间：256 MB

提示

[入栈和出栈都是操作着栈顶。]

[开一个大小为n的数组，记录栈顶的位置，入栈出栈就是将这栈顶加一减一，栈中某个位置Y在数组相应的下标就是Y。]

来自 <https://dsa.cs.tsinghua.edu.cn/oj/problem.shtml?id=2111>

代码实现:

```
#include <bits/stdc++.h>     //万能头
using namespace std;

const int N = 80005;

string Stack[N];  //C++字符数组，实现查询功能
int n,top=0; //全局变量top会自动初始化为0
//更新top位置
void push(string name) {
    Stack[top++] = name;
}
//对于void，可以写“return ;”，控制提前返回

string pop() {
    return Stack[--top];
}
//询问操作
string query(int pos) {
    return Stack[pos-1];
}

int main() {
    int n;
    scanf("%d", &n);
    char name[20];
    for (; n--; ) {
        int op;
        scanf("%d", &op);
        if (op == 1) {
            scanf("%s", name);
            push(name);
        } else if (op == 2) {
            printf("%s\n", pop().c_str());  //string的输出
        } else {
            int pos;
            scanf("%d", &pos);
            printf("%s\n", query(pos).c_str());
        }
    }
    return 0;
}
```

总结:

如何询问栈中某个位置的元素?

Stack不天然支持这个操作。

直接用数组下标访问即可。

Top不用初始化?

C++中，**全局变量**会调用构造函数**自动初始化**，这里top初始化为0。

@数据结构：数组+链表（巧妙！）

来自 <https://blog.csdn.net/cumtwyc/article/details/51638434>

2.队列

描述

实现一个队列，完成以下功能：

1. 入列
2. 出列
3. **询问队列中位置Y是谁**

一开始队列为空。队列中的位置从1开始（即队头位置为1）。

输入

第一行一个整数n，表示操作个数。

接下来n行，每行第一个数字表示操作（见描述）：

- 若为数字1，则接下来有一串字符串X，表示将X加入队列。
- 若为数字2，表示出列（保证队列非空），并输出出列的这个人。
- 若为数字3，则接下来有一个整数Y，表示询问队列中位置Y是谁（保证位置Y合法），并输出名字。

输出

将所有操作2和操作3输出，一行一个。

输入样例

```
11
1 a
1 b
1 c
3 1
3 2
3 3
2
1 d
3 1
3 2
```

```
3 3
```

输出样例

```
a
b
c
a
b
c
d
```

限制

对于30%的数据，1 ≤ n ≤ 2000；

对于另30%的数据，没有操作3；

对于100%的数据，1 ≤ n ≤ 100000。

数据中出现的字符串只包含26个小写字母（无空格等分隔符），且长度不超过15。

字符串有可能重复。正如现实中可能有重名一样。

时间：2 sec

空间：256 MB

提示

[队头出列，队尾入列。]

[开一个大小为n的数组，记录队头和队尾的位置，入列出列就是将这两个位置改变一下，队列中某个位置Y在数组相应的下标为队头的位置+Y-1。]（**操作3的解法**）

来自 <https://dsa.cs.tsinghua.edu.cn/oj/problem.shtml?id=2112>

代码实现:

```
#include <bits/stdc++.h>
using namespace std;

// ================== 代码实现开始 ==================

const int N = 10005;
string Queue[N];
int head=0,tail=0;

// 队尾入队
// name: 入队人的姓名
void enqueue(string name) {
```

```
    Queue[tail++] = name;
    //if（tail==N）tail==0;（类似于模操作）
}

// 队首出队
// 返回值：队首的姓名
string dequeue() {
    return Queue[head++];
    //if（head==N）head==0;
}

// 询问队列中某个位置上的人的姓名（队首位置为1，往后位置依次递增）
// pos：询问的位置
// 返回值：pos位置上人的姓名
string query(int pos) {
   return Queue[head-1 +pos];
    //if（p==N）p==0;    ## p=head-1 +pos
}

// ================= 代码实现结束 =================

int main() {
    int n;
    scanf("%d", &n);
    char name[20];
    for (; n--; ) {
        int op;
        scanf("%d", &op);
        if (op == 1) {
            scanf("%s", name);
            enqueue(name);
        } else if (op == 2) {
            printf("%s\n", dequeue().c_str());
        } else {
            int pos;
            scanf("%d", &pos);
            printf("%s\n", query(pos).c_str());
        }
    }
    return 0;
}
```

总结:

如何询问队列中某个位置的元素?

队列中某个位置Y在数组相应的下标haed+Y-1，head为队头的位置。

解法2:

入队元素次数超过k，队中元素不超过k。如何用大小为k的数组实现?

循环队列。（见代码中红色实现）

3.二叉树

描述

给定一个1到n的排列，按顺序依次插入到一棵**二叉排序树**中，请你将这棵二叉树前序遍历和后序遍历输出。

前序遍历的定义

后序遍历的定义

输入

第一行一个整数n。

接下来一行表示为n个整数，代表1到n的一个排列。

输出

输出所建成的二叉树的前序遍历和后序遍历。

输入样例

```
10
2 6 9 3 5 7 10 8 4 1
```

输出样例

```
2 1 6 3 5 4 9 7 8 10
1 4 5 3 8 7 10 9 6 2
```

限制

对于50%的数据，$1 \le n \le 100$；

对于100%的数据，$1 \le n \le 100000$。

保证建成的树的高度不超过50。

时间：2 sec

空间：256 MB

提示

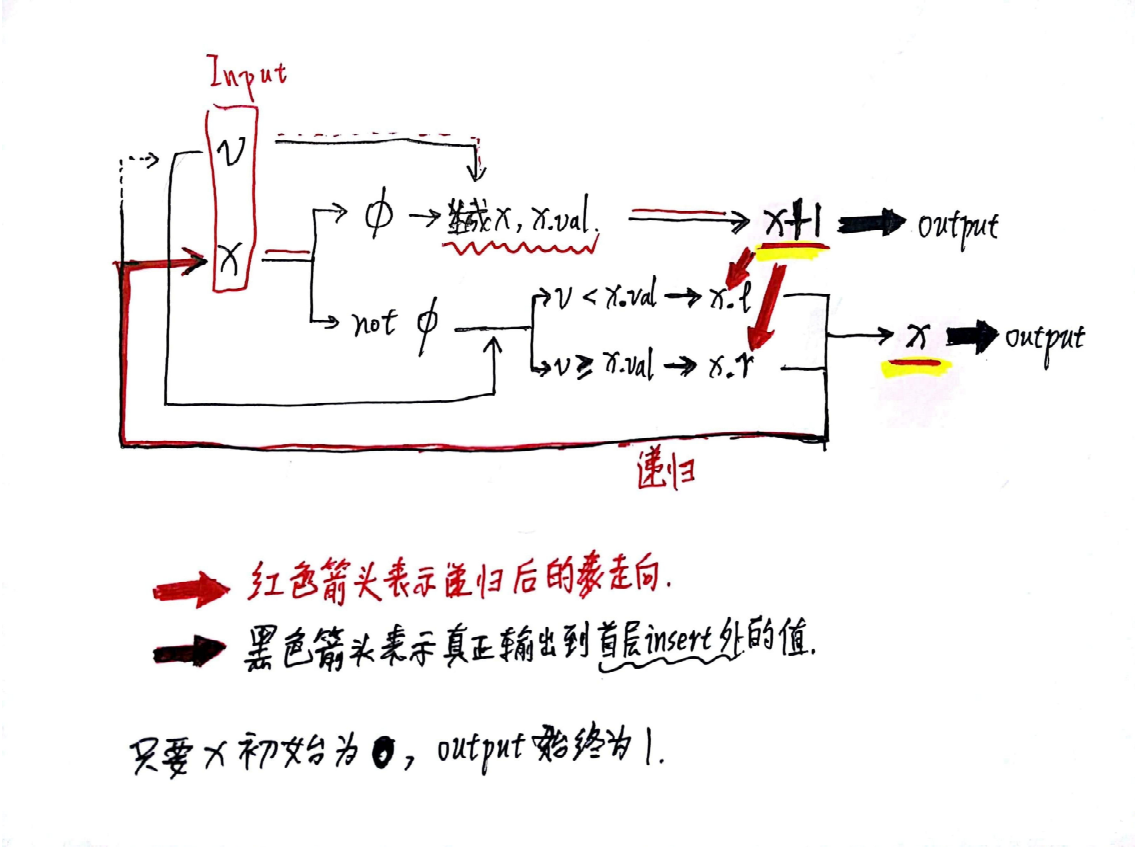
[二叉树的操作基本都是递归操作，只要想想如何在一个节点上判断是朝着左孩子走还是朝着右孩子走就行了。]

来自 <https://dsa.cs.tsinghua.edu.cn/oj/problem.shtml?id=2113>

代码实现:

```
#include <bits/stdc++.h>
using namespace std;

// ================== 代码实现开始 ==================

const int N = 100005;

//二叉树几点数据结构
//t: 二叉树节点数组
struct node{
    int val,l,r;
}t[N];//下标当作指针使用，t为元素为node的一个结构体数组

//root: 整个二叉树的根节点
//cnt: 整个二叉树的大小
int root,cnt;  //这里已经自动初始化为0了

int insert(int v,int x){

    if(x == 0){
        //当前节点不存在，更新节点（仅根）
        root=1; //root不能为始终为0
        x= ++cnt;
        t[x].val=v;
        t[x].l=0;
        t[x].r=0;
        return x;
    }
    //递归插入左右子树
    if(v < t[x].val) {
        t[x].l=insert(v,t[x].l);
    }
    else{
        t[x].r=insert(v,t[x].r);
      }
    return x;
```

```
}
//前序，根、左、右
void dlr(int x,vector<int> &ans){
	if(x){
		ans.push_back(t[x].val);
		//cout<<t[x].val<<" ";
		dlr(t[x].l,ans);
		dlr(t[x].r,ans);
	}

}
//后序，左、右、根
void lrd(int x,vector<int> &ans){
	if(x||t[x].val){
		lrd(t[x].l,ans);
		lrd(t[x].r,ans);
		ans.push_back(t[x].val);
		//cout<<t[x].val<<" ";
	}
}

// 给定一个1到n的排列，依次插入到二叉树中，返回前序遍历和后序遍历
// n：如题意
// sequence：给定的排列，大小为n
// 返回值：将要输出的元素依次加入到返回值中
vector<int> getAnswer(int n, vector<int> sequence) {
   root=cnt=0; //初始化 ，无必要，之前全局变量已自动为0了
   //将序列插入二叉树
	for(int i = 0; i < int(sequence.size());++i)
		insert(sequence[i],root);
		//root = insert(sequence[i],root);随时更新root（疑似过于“当地”）
		//正确！如下图解释所示，insert的第1层返回值始终为“1”，root不动，恒等于
		1
		//图中“x+1”准确应为“++cnt”
```




```
    vector<int> ans; //返回值
  dlr(root,ans);
  lrd(root,ans);
  return ans; //这里是坑！！！把这里写掉了，程序仍然会运行，只是会突然崩溃！
}

// ================= 代码实现结束 =================

int main() {
  int n;
  scanf("%d", &n);
  vector<int> sequence;
  for (int i = 0; i < n; ++i) {
    int x;
    scanf("%d", &x);
    sequence.push_back(x);
  }
  vector<int> ans = getAnswer(n, sequence);
    for (int i = 0; i < n; ++i)
    printf("%d%c", ans[i], " \n"[i == n - 1]);
//%c表示将输出一个字符；" \n"相当于一个字符数组，[i==n-1]当真时取1，即"\n"；假取0，则得到" "空格符
  for (int i = 0; i < n; ++i)
    printf("%d%c", ans[n + i], " \n"[i == n - 1]);
  return 0;
```

}

总结:

排列：无重复元素

深度≤50：插入的时间复杂度O(50n)

前序遍历：**根**-左-右，O(n)

后序遍历：左-右-**根**，O(n)